Intelligent Robotics (ASY2) · Lab 2

Polygon map

Least squares and RANSAC on a laser scan

Duration	180 minutes	Work in	pairs
Script	Cycle 2, section 2.2.3	Files	`ohm-polygon-map/student/`

1 Overview

A robot stands in a room. We know two things about the room: it is a rectangle, and there is one round object in it. From a single laser scan you build a map of the room:

- the four walls as straight lines,
- the four corners as the intersections of these lines,
- the round object as a circle.

One corner is hidden behind the object (fig. 1). No beam reaches it, but your map will contain it anyway.

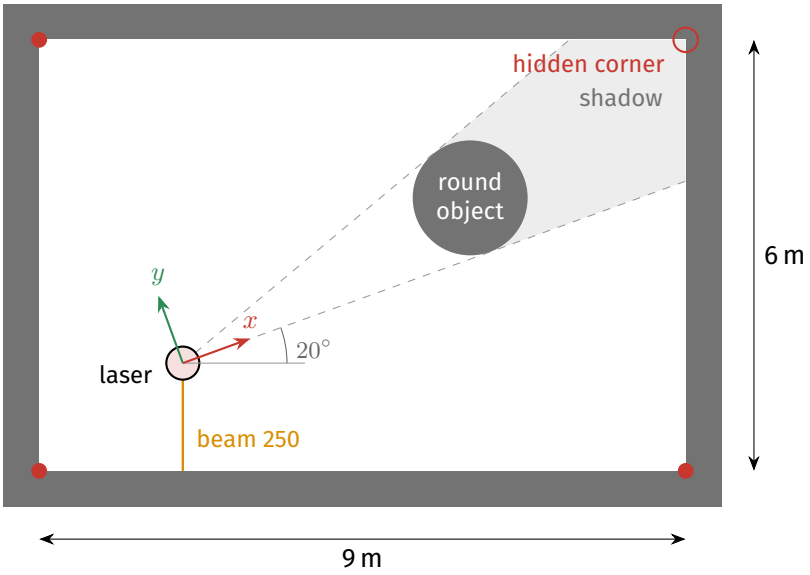


Figure 1: The hall `box` from above. The laser looks along x; all points, lines and circles in this lab are in its frame.

Notation as in the script: a point is $(\hat{x}_i, \hat{y}_i)$, a line is $y = \tilde{m}\,x + \tilde{t}$, a circle has the centre $(\tilde{x}_0, \tilde{y}_0)$ and the radius $\tilde{r}$. In the code: points `(x, y)`, lines `(m, t)`, circles `(x0, y0, r)`, all in metres in the frame of the laser. The equations you need are repeated in the tasks, with the numbers they have in the script.

Table 1: Schedule.

min	task	done when
0–20	section 3 The scan and its noise	table 2 is filled in
20–60	section 4 RANSAC: one line	a red line lies on a wall
60–85	section 5 Least squares	the refined line fits better
85–95	break	
95–120	section 6 Four lines	four walls in four colours
120–140	section 7 Corners	a red rectangle, 9×6 m
140–165	section 8 The circle	a red circle on the object
165–180	section 9 Extra tasks	

2 Getting started

Your own copy. Fork `ohm-polygon-map` on GitHub (button *Fork* at the top right), then clone your fork into the `src/` folder of your workspace. The simulator is still there from lab 1; update it:

```
git clone git@github.com:<your-name>/ohm-polygon-map.git
```

```
git -C ir-mecanum-lab pull
```

Commit after every task and push, so nothing is lost at the end of the session.

Build. Build once, then source the workspace in every new terminal:

```
colcon build --symlink-install --packages-select mecanum_lab_interfaces mecanum_lab
    ohm_polygon_map
```

```
source install/setup.bash
```

You do not need to build again after a change to your code. Just start the node again.

Simulator. One launch file starts the simulator in the hall `box` and RViz. The robot stands still; the keys do not move it.

```
ros2 launch ohm_polygon_map box.launch.py
```

Your files. `student/line_fitting.py` (tasks 0 to 4) and `student/circle_fitting.py` (task 5), plain Python without ROS. In every function the checks (too few points, `None`, parallel lines, …) are already there. You write only the few lines in the middle, marked

```
# m = (y2 - y1) / (x2 - x1),   then t from  y1 = m x1 + t
# TODO STUDENT 1.1
raise NotImplementedError("STUDENT 1.1")
```

The comment above it says what to compute. Replace the `raise` line by your code. Use only Python and `math`, no `numpy`. The helpers you call are given in `ohm_polygon_map/geometry.py`, the drawing in RViz in `ohm_polygon_map/rviz_draw.py`. You do not change these files.

The node. `student/polygon_map_node.py` is given. It subscribes to the scan, calls your functions and draws the result in RViz. The parameter `stage` says how far it goes, from 0 (the noise) to 5 (the circle). It runs in a second terminal:

```
ros2 run ohm_polygon_map polygon_map_node.py --ros-args -p stage:=1
```

Every 2 s it takes the newest scan and prints what it found. If a function is not written yet, the node stops with `NotImplementedError: STUDENT 1.1`: that is the next one to write.

Self check. The tests try your functions on made-up points, one stage at a time:

```
python3 -m pytest test/test_fitting.py -q -k stage1
```

Three terminals. All of them in the workspace, sourced.

terminal 1	simulator and RViz	stays open the whole session
terminal 2	your node	Ctrl-C and start again after every change
terminal 3	tests, hints, `ros2 topic`	

Every task starts with a box *Running*: what terminals 1 and 2 run. It ends with *What you should see*: RViz and terminal 2 when the task is done. RViz shows only the scan and what your node draws, not the walls of the hall. The numbers change a little from scan to scan.

Hints. Stuck? Every block has three hints: the idea, how to write it in Python, and at last the code. Each call opens the next one, in the folder `ohm-polygon-map`:

```
python3 hint.py 1.3
```

`python3 hint.py` alone shows the hints you have opened. Try on your own first: the hints notice when you ask too early.

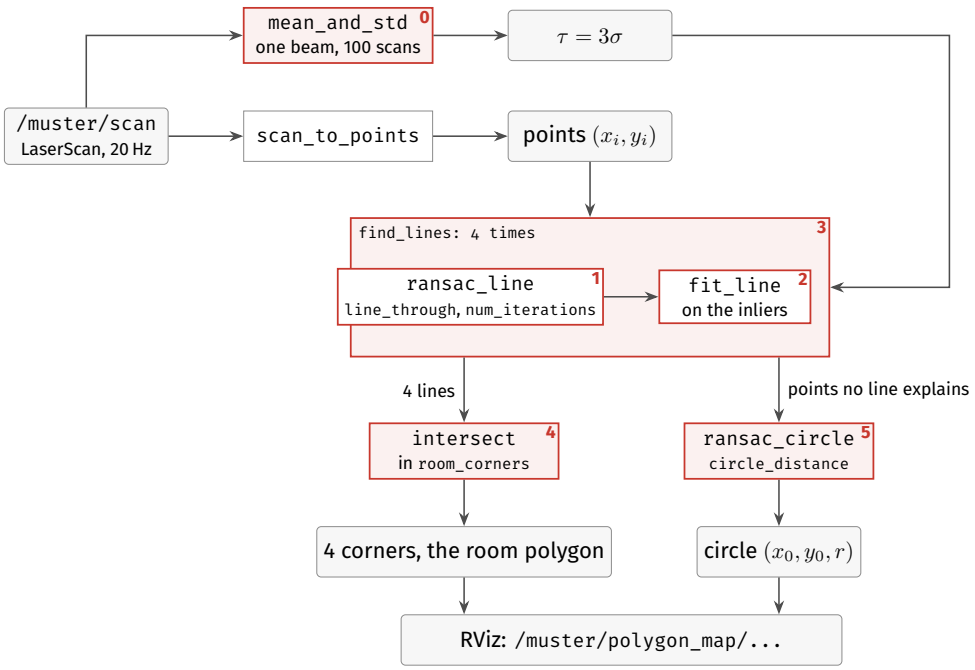


Figure 2: From the scan to the map. Red: you write it (the number is the task). Grey: given.

3 Task 0: the scan and its noise

20 min

Running

```
terminal 1   ros2 launch ohm_polygon_map box.launch.py
terminal 2   ros2 run ohm_polygon_map polygon_map_node.py --ros-args -p stage:=0
```

1. In terminal 3, look at one scan:

   ```
   ros2 topic echo /muster/scan --once
   ```

   Fill in the first rows of table 2. How many beams are there? In which direction does beam 0 point, in which beam 90?

2. Read `scan_to_points` in `geometry.py`. Which beams does it drop, and why?

A wall that does not move should give the same range in every scan. It does not: the laser is noisy. You measure how noisy, because RANSAC needs a threshold τ for "close to the line".

Lecture script, section 2.2.3, the one-dimensional case

The least-squares estimate of a constant from the readings $\hat{x}_1 \dots \hat{x}_n$ is their mean:

$$\tilde{c} = \frac{1}{n} \sum_{i=1}^{n} \hat{x}_i. \tag{2.18}$$

STUDENT 0.1 `line_fitting.py`, `mean_and_std(values)`: the mean (2.18) and the sample standard deviation $\sigma = \sqrt{\frac{1}{n-1} \sum_i (\hat{x}_i - \tilde{c})^2}$.

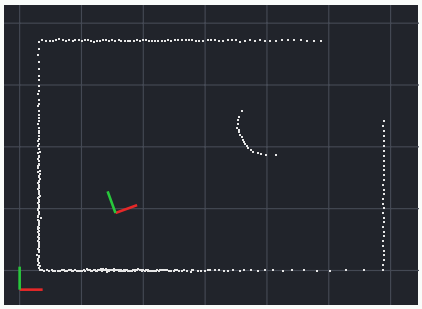
`python3 hint.py 0.1`

Run the node with `stage:=0`. It collects beam 250 over 100 scans (5 s), which hits the lower wall at a right angle (fig. 1). It prints the mean, σ and $\tau = 3\sigma$. Repeat with another beam, e.g. `-p noise_beam:=90`.

Table 2: The scan and its noise. Fill in.

	beam 250	beam 90
`angle_increment`, number of beams		
`range_max`, a beam without echo		
mean range $\tilde{c}$		
σ		
$\tau = 3\sigma$		

What you should see



tick `raw scan` in the list of displays of RViz; terminal 2:

```
[0] beam 250 over 100 scans: mean 1.4992 m, sigma 14.6 mm ->
    tau = 3 sigma = 43.8 mm
```

Checkpoint σ is about the same for both beams. Why 3σ? What share of the points on a wall lies closer than τ to it?

4 Task 1: RANSAC, one line

40 min

Running

```
terminal 1   ros2 launch ohm_polygon_map box.launch.py
terminal 2   ros2 run ohm_polygon_map polygon_map_node.py --ros-args -p stage:=1
```

A scan of the room has points on four walls and on the object. A least-squares line through all of them fits nothing. RANSAC turns the question around: draw two points, put a line through them, and count how many points agree with it.

Lecture script, section 2.2.3, robust fitting: RANSAC

Repeat k times: *sample* the minimum number of points (two for a line), *fit* the model, *score* it by its inliers, the points closer than τ. *Keep* the model with the most inliers. To draw at least one all-inlier sample with probability p, with inlier ratio w and sample size s:

$$k = \frac{\log(1-p)}{\log\left(1-w^s\right)}. \tag{2.30}$$

STUDENT 1.1 `line_through(p, q)`: the line $y = mx + t$ through two points, $m = (y_2 - y_1)/(x_2 - x_1)$, $t = y_1 - m\,x_1$. Two points above each other are already caught.

`python3 hint.py 1.1`

STUDENT 1.2 `num_iterations(w, s, p)` with (2.30), rounded up (`math.ceil`). Check with the exercise of the script: $w = 0.8$ gives 5, $w = 0.5$ gives 17. `python3 hint.py 1.2`

Given in `geometry.py`: `line_distance(point, line)` is the distance perpendicular to the line, $d = |mx - y + t|/\sqrt{1+m^2}$, and `split_line(points, line, tau)` returns `(inliers, outliers)`. `rng.sample(points, 2)` draws two different points.

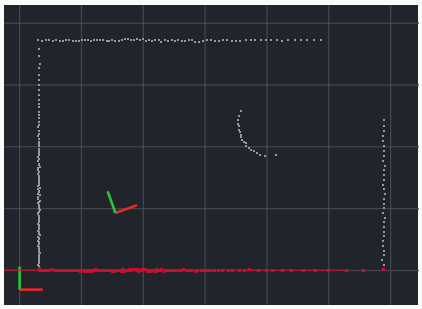
STUDENT 1.3 `ransac_line(points, tau, iterations, rng)`: the loop already draws two points and puts a line through them (*sample*, *fit*). You write *score* and *keep*: the inliers of the line, and if there are more than for the best line so far, this line becomes the best line.

`python3 hint.py 1.3`

Run the node with `stage:=1`. RViz shows the scan in grey, the inliers and the line in red. The node uses $w = 0.3$ and $p = 0.99$.

- Which wall does RANSAC find? Why that one?
- Run it with `-p seed:=2` up to `seed:=5`. Does the result change?
- Set `-p w_line:=0.9`. How many iterations is that? What happens?

What you should see



terminal 2:

```
[1] 360 points, k = 49: RANSAC line y =  -0.364 x + -1.587 (
    -20.0 deg), 133 inliers
```

Checkpoint `pytest -k stage1` passes. In RViz the red line lies on a wall.

5 Task 2: least squares

25 min

Running

```
terminal 1   ros2 launch ohm_polygon_map box.launch.py
terminal 2   ros2 run ohm_polygon_map polygon_map_node.py --ros-args -p stage:=2
```

The RANSAC line goes through two noisy points. All its inliers together say more: fit a line to them by least squares.

Lecture script, section 2.2.3, the two-dimensional case

The line $y = \tilde{m}x + \tilde{t}$ that minimises the squared error in y, $\epsilon = \sum_i \left(\hat{y}_i - (\tilde{m}\,\hat{x}_i + \tilde{t})\right)^2$, has

$$\tilde{m} = \frac{\sum_{i=1}^{n} (\hat{x}_i - \overline{\hat{x}})(\hat{y}_i - \overline{\hat{y}})}{\sum_{i=1}^{n} (\hat{x}_i - \overline{\hat{x}})^2}, \tag{2.23}$$

$$\tilde{t} = \overline{\hat{y}} - \tilde{m}\,\overline{\hat{x}}. \tag{2.24}$$

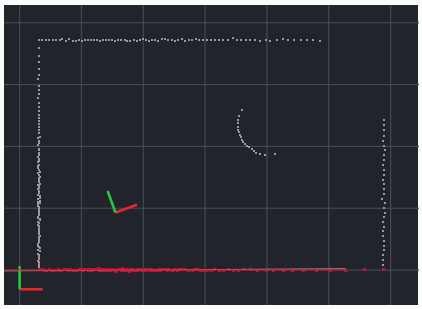
STUDENT 2.1 `fit_line(points)`: $\tilde{m}$ with (2.23) and $\tilde{t}$ with (2.24). The means $\overline{\hat{x}}$, $\overline{\hat{y}}$ are already computed. `python3 hint.py 2.1`

`pytest -k stage2` also tries the exercise of the script: the points $(0, 1.1)$, $(1, 2.9)$, $(2, 5.2)$, $(3, 6.8)$ give $\tilde{m} = 1.94$ and $\tilde{t} = 1.09$.

The node fits your line to the inliers of the RANSAC line and then takes the inliers of the new line. Run it with `stage:=2`: grey is the RANSAC line, red the least-squares line.

- Compare m and t of both lines over a few evaluations. Which one changes less?
- Has the number of inliers changed?

What you should see



terminal 2:

```
[1] 360 points, k = 49: RANSAC line y =  -0.357 x + -1.596 (
    -19.6 deg), 132 inliers
[2] least squares y =  -0.364 x + -1.595 ( -20.0 deg), 132
    inliers
```

Checkpoint `pytest -k stage2` passes. The least-squares line is steadier from scan to scan than the RANSAC line.

6 Task 3: four lines

25 min

Running

```
terminal 1   ros2 launch ohm_polygon_map box.launch.py
terminal 2   ros2 run ohm_polygon_map polygon_map_node.py --ros-args -p stage:=3
```

Each wall is an outlier for the other three walls. So after one wall is found, its inliers are removed, and RANSAC runs again on the rest: *sequential RANSAC*.

STUDENT 3.1 `find_lines(points, n, tau, iterations, min_inliers, rng)` repeats n times:

1. `ransac_line` on the remaining points (given),
2. stop if it has fewer than `min_inliers` (given),
3. `fit_line` on its inliers,
4. `split_line` of the remaining points with this line: the inliers belong to the wall, the outliers are the new remaining points,
5. append `(line, inliers)` to `lines`.

You write steps 3 to 5, three lines. `python3 hint.py 3.1`

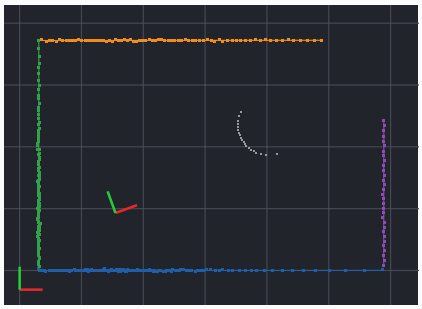
Run the node with `stage:=3`. Each wall gets its own colour. Fill in table 3: N is the number of points at the start of the round (round 1: all points, the node prints them; then subtract the inliers of the round before), n the inliers of the line found, $w = n/N$, and k the iterations (2.30) would ask for with this w.

Table 3: Sequential RANSAC, round by round. Fill in.

round	wall	N	n	$w = n/N$	k
1					
2					
3					
4					

- The node uses $k = 49$ ($w = 0.3$) in every round. Too many, too few?
- Why does one wall have far fewer points than the others? (fig. 1)
- What would happen without `min_inliers`, if you asked for five lines?

What you should see



terminal 2:

```
[3] 360 points: 4 lines (k = 49 each), 20 points left over
    wall 1: y =  -0.364 x + -1.595 ( -20.0 deg), 133 inliers
    wall 2: y =   2.745 x +  5.848 (  70.0 deg), 102 inliers
    wall 3: y =  -0.363 x +  4.788 ( -20.0 deg), 74 inliers
    wall 4: y =   2.743 x + -20.435 (  70.0 deg), 31 inliers
```

Checkpoint `pytest -k stage3` passes. Four walls in four colours, about 20 points left over.

7 Task 4: corners

20 min

Running

terminal 1 `ros2 launch ohm_polygon_map box.launch.py`
terminal 2 `ros2 run ohm_polygon_map polygon_map_node.py --ros-args -p stage:=4`

Two walls meet in a corner: where both line equations hold,

$$m_1 x + t_1 = m_2 x + t_2 \quad \Rightarrow \quad x = \frac{t_2 - t_1}{m_1 - m_2}, \qquad y = m_1 x + t_1. \tag{1}$$

Four lines make six pairs. Two of them are opposite walls: parallel, no corner.

STUDENT 4.1 `intersect(l1, l2, min_angle)`: x and y with eq. (1). Parallel lines (an angle below `min_angle`) are already caught. `python3 hint.py 4.1`

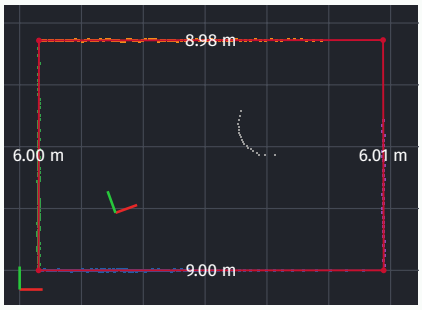
The given `room_corners(lines, intersect)` calls your `intersect` on all pairs, drops the `None`s and orders the corners around their centre, so that they form a polygon. Run the node with `stage:=4`. RViz draws the room polygon in red with the length of each side; the node also prints the angles at the corners.

Table 4: The room. Fill in.

	side 1	side 2	side 3	side 4
length in m				
angle at its first corner				

- Which corner was never measured? How far is it from the nearest scan point?
- Why is `min_angle` needed, and not `m1 == m2`?

What you should see



terminal 2:

```
[3] 360 points: 4 lines (k = 49 each), 20 points left over
    wall 1: y =  -0.364 x + -1.597 ( -20.0 deg), 133 inliers
    wall 2: y =   2.737 x +  5.836 (  69.9 deg), 102 inliers
    wall 3: y =  -0.363 x +  4.785 ( -19.9 deg), 74 inliers
    wall 4: y =   2.770 x + -20.597 (  70.2 deg), 31 inliers
[4] 4 corners: (-2.397, -0.725), (6.062, -3.803), (8.102,
   1.848), (-0.339, 4.908)
    sides 9.002, 6.008, 8.979, 5.997 m, angles 89.9, 89.9,
   90.1, 90.1 deg
```

Checkpoint `pytest -k stage4` passes. The sides are 9 and $6\,\mathrm{m}$ within a few centimetres, the angles $90°$ within a few tenths of a degree.

8 Task 5: the circle

25 min

Running

```
terminal 1   ros2 launch ohm_polygon_map box.launch.py
terminal 2   ros2 run ohm_polygon_map polygon_map_node.py --ros-args -p stage:=5
```

After the four walls, about 20 points are left: the object. RANSAC works for a circle as it does for a line. Only three things change.

Lecture script, section 2.2.3, example: RANSAC for circles

Sample: $s = 3$ points; three points not on a line define one circle. *Score*: point i is an inlier if $\left|\sqrt{(\hat{x}_i - \tilde{x}_0)^2 + (\hat{y}_i - \tilde{y}_0)^2} - \tilde{r}\right| < \tau$. *Refit*: the least-squares circle (2.37), (2.38) on all inliers. Applied to three points, the same formulas return the circle through them.

Given in `geometry.py`: `fit_circle(points)` is (2.37) and (2.38); it returns `None` for points on a line. Your file this time: `student/circle_fitting.py`.

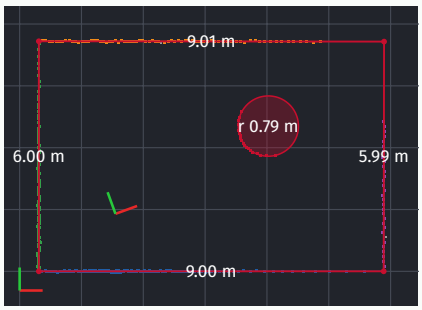
STUDENT 5.1 `circle_distance(point, circle)`: the distance of the *score* in the script box. `python3 hint.py 5.1`

STUDENT 5.2 `ransac_circle(points, tau, iterations, rng, r_max)`: as in `ransac_line`, the loop already draws three points and fits the circle through them; circles larger than `r_max` are skipped (three points almost on a line give a huge circle). You write *score* and *keep*, with `circle_distance`. The refit on all inliers at the end is given. `python3 hint.py 5.2`

Run the node with `stage:=5`. The node uses $w = 0.5$, so $k = 35$.

- Centre and radius of the circle? Compare with `config/box.json` (hall coordinates, so it does not match directly: why?).
- How many iterations would (2.30) need on the *whole* scan, $w = 20/360$, $s = 3$? What does that say about the order of tasks 3 and 5?

What you should see



terminal 2:

```
[3] 360 points: 4 lines (k = 49 each), 22 points left over
    wall 1: y =  -0.364 x + -1.595 ( -20.0 deg), 132 inliers
    wall 2: y =   2.747 x +  5.846 (  70.0 deg), 103 inliers
    wall 3: y =  -0.365 x +  4.791 ( -20.1 deg), 73 inliers
    wall 4: y =   2.733 x + -20.371 (  69.9 deg), 30 inliers
[4] 4 corners: (-2.392, -0.725), (6.062, -3.801), (8.121,
   1.825), (-0.339, 4.915)
    sides 8.996, 5.991, 9.006, 6.001 m, angles 90.0, 90.1,
   90.0, 89.9 deg
[5] circle centre (4.530, 0.787), r = 0.787 m, 20 of 22
   points (k = 35)
```

Checkpoint `pytest -k stage5` passes. A red circle lies on the object, the radius is $0.8\,\mathrm{m}$ within a few centimetres.

9 Extra tasks

Z.1: a vertical wall

Stop the simulator and start it with the robot turned to $0°$:

Running

```
terminal 1   ros2 launch ohm_polygon_map box.launch.py start:=straight
terminal 2   ros2 run ohm_polygon_map polygon_map_node.py --ros-args -p stage:=4
             [-p hesse:=true]
```

Run the node with `stage:=4`. What happens to the map? Look at m of the walls that run along y. The script names the problem: $y = mx + t$ measures the error only in y, and it cannot describe a vertical line. Robots use the Hesse normal form $x\cos\alpha + y\sin\alpha = r$ instead.

Lecture script, section 2.2.3, orthogonal least squares and the Hesse normal form

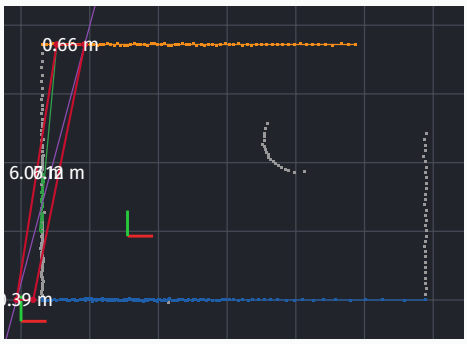
$$q_x = \sum_{i=1}^{n}(\hat{x}_i - \overline{\hat{x}})^2, \qquad q_y = \sum_{i=1}^{n}(\hat{y}_i - \overline{\hat{y}})^2, \qquad q_{xy} = \sum_{i=1}^{n}(\hat{x}_i - \overline{\hat{x}})(\hat{y}_i - \overline{\hat{y}}), \tag{2.25}$$

$$\alpha = \tfrac{1}{2}\,\mathrm{atan2}(-2\,q_{xy},\; q_y - q_x)\,, \qquad r = \overline{\hat{x}}\cos\alpha + \overline{\hat{y}}\sin\alpha. \tag{2.29}$$

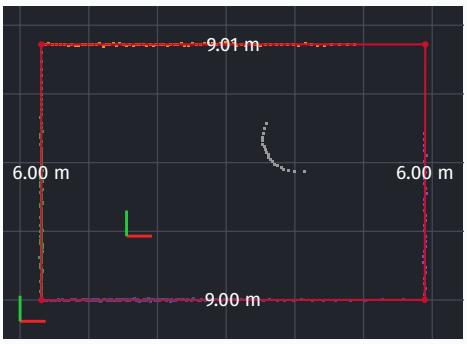
If r comes out negative, add π to α and flip the sign of r.

STUDENT Z.1 `fit_line_hesse(points)` in `line_fitting.py`: q_x, q_y, q_{xy} with (2.25), then α and r with (2.29). The means and the sign of r are already handled. Test with `pytest -k extra`, then run the node with `-p stage:=4 -p hesse:=true`: tasks 3 and 4 now use your function instead of `fit_line`. `python3 hint.py Z.1`

What you should see



`-p hesse:=false`, $y = mx + t$



`-p hesse:=true`, your `fit_line_hesse`

terminal 2:

```
# -p hesse:=false
[4] 4 corners: (-2.605, -1.497), (-2.214, -1.497), (-1.006, 4.500), (-1.663, 4.500)
    sides 0.392, 6.117, 0.657, 6.070 m, angles 81.1, 101.4, 78.6, 98.9 deg
# -p hesse:=true
[4] 4 corners: (-1.998, -1.501), (7.001, -1.496), (7.014, 4.501), (-2.001, 4.500)
    sides 8.999, 5.997, 9.015, 6.001 m, angles 90.0, 90.2, 89.9, 90.0 deg
```

Why does RANSAC still find the vertical walls with `line_through`, which only gives m and t?

Z.2: how exact is the circle?

The node evaluates a new scan every 2 s. Note the radius of ten evaluations and compute its mean and σ with your `mean_and_std`. Then compare with the circle seen from closer: how would you expect σ of the radius to change with the distance to the object, and why?